

Q1

10 Points

The questions are based on the following C code and their corresponding compiled code:

```
void incr(int *p)
{
    (*p)++;
}

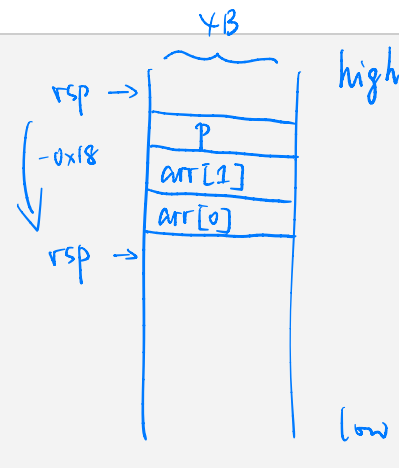
void test()
{
    1: int p = 16;
    2: int arr[2] = {32,48};
    3: incr(&p);
    4: incr(&arr[1]);
    5: incr(&arr[2]);
    6: printf("%d,%d,%d\n", p, arr[0], arr[1]);
}

int main()
{
    test();
}
```

Procedure `incr`, disassembled using gdb. Note the first column denote the instruction's address.

0x0000000000401106 <+0>:	mov	__(1)___	%eax = *p → (%rdi), %eax
0x0000000000401108 <+2>:	add	\$0x1,%eax	%eax++
0x000000000040110b <+5>:	mov	__(2)___	*p = %eax
0x000000000040110d <+7>:	ret		%eax, (%rdi)

Procedure `test`, disassembled using gdb.

0x000000000040110e <+0>:	sub	\$0x18,%rsp	
0x0000000000401112 <+4>:	movl	\$0x10,0xc(%rsp)	
0x000000000040111a <+12>:	movl	\$0x20,0x4(%rsp)	
0x0000000000401122 <+20>:	movl	\$0x30,0x8(%rsp)	
0x000000000040112a <+28>:	lea	0xc(%rsp),%rdi	
0x000000000040112f <+33>:	call	0x401106 <incr>	
0x0000000000401134 <+38>:	lea	0x8(%rsp),%rdi	
0x0000000000401139 <+43>:	call	0x401106 <incr>	
0x000000000040113e <+48>:	lea	0xc(%rsp),%rdi	
0x0000000000401143 <+53>:	call	0x401106 <incr>	
0x0000000000401148 <+58>:	add	\$0x10,%rsp	
....omitted lines for invoking printf.....			
0x000000000040114c <+62>:	ret		

Procedure `main`, disassembled using gdb.

```

0x000000000040114d <+0>:    mov     $0x0,%eax
0x0000000000401152 <+5>:    call   0x40110e <test>
0x0000000000401157 <+10>:   mov     $0x0,%eax
0x000000000040115c <+15>:   ret

```

Q1.1

1 Point

What is the missing instruction in procedure `incr` at `__(1)__`? For autograding, omit the operation name `mov` and ensure there's no space in your answer.

`(%rdi),%eax`

Q1.2

1 Point

What is the missing instruction in `incr` at `__(2)__`? For autograding, omit the operation name `mov` and ensure there's no space in your answer.

`%eax,(%rdi)`

Q1.3

1 Point

Suppose the value of `%rsp` is `0x7fff856001d8` **just prior to** executing the first instruction of `test`. What is the value of `%rsp` **just prior to** executing the first instruction of `incr` after invoking `incr` at line 4?

`0x7fff856001d8`

`0x7fff856001c8`

`0x7fff856001e8`

`0x7fff856001c0`

✓ `0x7fff856001b8`

`0x7fff856001b0`

`0x7fff856001cc`

None of the above

return address already pushed

allocate 0x18 B space for <test>

call incr: push return addr to stack (8B)

*$0x7fff856001d8 - 0x18 - 0x8$
 $= 0x \dots 1b8$*

Q1.4

1 Point

Continuing from Q1.3, **just prior** to executing the first instruction of `incr` after `test` invokes `incr` at line 4, what is the 8-byte memory content whose starting address is in `%rsp`? aka, what are the 8-byte stored on the top of the stack?

0x00007fff856001c0
0x00007fff856001d8
0x000000000040110d
0x000000000040112a
0x000000000040112f
0x0000000000401134
0x0000000000401139
✓ 0x000000000040113e
0x0000000000401143
0x0000000000401148

return addr : next instruction after func return

```
0x000000000040110e <+0>:    sub    $0x18,%rsp
0x0000000000401112 <+4>:    movl   $0x10,0xc(%rsp)
0x000000000040111a <+12>:   movl   $0x20,0x4(%rsp)
0x0000000000401122 <+20>:   movl   $0x30,0x8(%rsp)
0x000000000040112a <+28>:   lea    0xc(%rsp),%rdi
0x000000000040112f <+33>:   call   0x401106 <incr>
0x0000000000401134 <+38>:   lea    0x8(%rsp),%rdi
0x0000000000401139 <+43>:   call   0x401106 <incr>
0x000000000040113e <+48>:   lea    0xc(%rsp),%rdi
0x0000000000401143 <+53>:   call   0x401106 <incr>
0x0000000000401148 <+58>:   add    $0x10,%rsp
....omitted lines for invoking printf.....
0x000000000040114c <+62>:   ret
```



Q1.5

1 Point

Where is procedure `test`'s local variable `p` stored?

- some register
- memory (data segment)
- ✓ memory (stack)
- memory (heap)

Q1.6**1 Point**

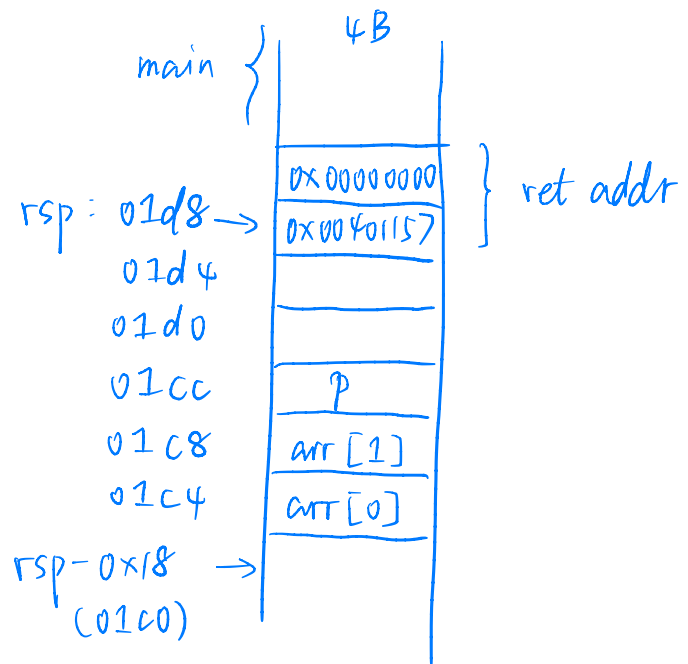
Suppose the value of `%rsp` is `0x7fff856001d8` **just prior to** executing the first instruction of `test`. What is the address of array `arr[0]` allocated by `test`? To facilitate autograding, start your answer (a hex value) with prefix `0x` and omit any leading 0s.

0x7fff856001c4

Q1.7**1 Point**

Suppose the value of `%rsp` is `0x7fff856001d8` **just prior to** executing the first instruction of `test`. What is the address of array `arr[1]` allocated by `test`? To facilitate autograding, start your answer (a hex value) with prefix `0x` and omit any leading 0s.

0x7fff856001c8



Q1.8

1 Point

Suppose the value of `%rsp` is `0x7fff856001d8` **just prior to** executing the first instruction of `test`. What can be said about memory address `0x7fff856001cc`? (Circle all that apply)

nothing of value (just a gap) is stored at that address

`test`'s local variable `p` is stored at that address

it is (address) value of expression `&arr[2]`

saved return address `0x0000000000401134` is stored at that address

saved return address `0x000000000040113e` is stored at that address

saved return address `0x0000000000401148` is stored at that address

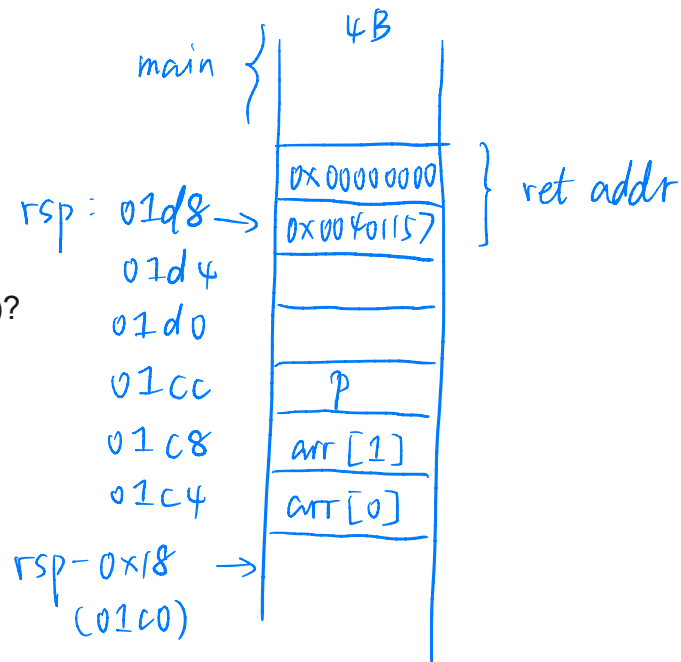
saved return address `0x0000000000401157` is stored at that address

Q1.9

1 Point

What is the output of the program (aka the printout of line 7)?

18,32,49



Q1.10
1 Point

Which of the following change at line 5 will cause the program attempt to execute instruction at address `0x0000000000401158` and causes a segmentation fault because no valid x86 instruction can be found starting at that address?

`incr(&arr[0])`

`incr(&arr[1])`

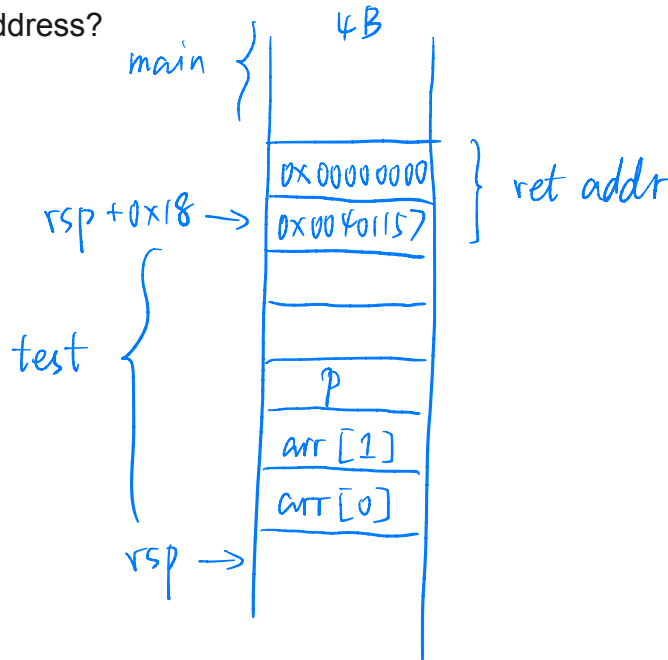
`incr(&arr[3])`

`incr(&arr[4])`

☒ `incr(&arr[5])`

`incr(&arr[6])`

none of the above



`incr(&arr[6])`

`0x0000000000401157`